Assignement 4
Neural networks and PSO learning

Mael Madec

May 25, 2025

Contents

1. Introduction 3
2. Theoretical Background 4
   2.1. Overview of Neural Networks 4
      2.1.1. Structure of a Feedforward Neural Network 4
      2.1.2. Training the Network: 4
   2.2. Overview of PSO Optimization 5
      2.2.1. Mathematical formulation 5
3. Implementation 6
   3.1. Data Preparation 6
   3.2. Classification 6
   3.3. Neural Network Setup 6
   3.4. Neural Network training 6
   3.5. PSO Optimization 7
4. Results 9
   4.1. Feedforward neural network 9
   4.2. Particle Swarm Optimization 10
   4.3. Linear activation neural network 10
      4.3.1. Implementation 10
5. Conclusion 12
Bibliography 13
Index of Figures 14

1. Introduction

This report presents an analysis of 2 classification methods applied to a breast cancer dataset, with a feedforward neural networks and a Particle Swarm Optimization algorithm (PSO). The goal is to evaluate their accuracy, training mechanisms, and overall effectiveness in binary classification tasks.

2. Theoretical Background

2.1. Overview of Neural Networks

A feedforward neural network is a type of artificial neural network where the connections between the nodes are moving in only one direction—forward—from the input nodes, through the hidden nodes (if any), and to the output nodes.

2.1.1. Structure of a Feedforward Neural Network

1. Input layer: The input layer is the first layer of the network and is responsible for receiving the input data. Each node in the input layer represents a feature or variable from the input data [1]. In this dataset, it means that we have 30 input nodes.

2. Hidden layer: Hidden layers are intermediate layers between the input layer and the output layer. A feedforward neural network can have one or more hidden layers. Each hidden layer consists of a certain number of neurons or nodes.

   Each neuron in a hidden layer is connected to every neuron in the previous layer and the subsequent layer. These connections have associated weights, which are adjusted during the training process.

   Neurons in hidden layers apply an activation function to the weighted sum of their inputs to introduce non-linearity into the model. Common activation functions include ReLU (Rectified Linear Unit), sigmoid, and tanh [1].

   The number of hidden layers and the number of neurons in each layer are hyperparameters that can be tuned to optimize the performance of the network. More hidden layers and neurons can capture more complex patterns but may also lead to overfitting.

3. Output layer:

   The output layer is the final layer of the network and is responsible for producing the output of the network. The number of nodes in the output layer depends on the type of problem being solved. For example, a binary classification problem will have one output node, while a multi-class classification problem will have multiple output nodes.

   Similar to the hidden layers, the output layer applies an activation function to produce the final output. The choice of activation function depends on the type of problem (e.g., sigmoid for binary classification, softmax for multi-class classification) [1].

2.1.2. Training the Network:

The training process involves adjusting the weights and biases of every layers to minimize the error between the predicted output and the actual output. In this implementation, the optimization algorithm is the Levenberg-Marquardt which provides a numerical solution to the problem of minimizing a function, which is often nonlinear and dependent on several variables.

Cross-validation is one additional techniques used in training the network. The original dataset containing N data is divided into S subsets. S – 1 subsets are used for learning, and one for validation of the model. This procedure is then repeated for all possible S combinations.

In this way we can use much more original data for learning purposes. [2] Cross-validation in this context allows for more robust and reliable estimation from our limited dataset.

Additionally, the error is propagated backward through the network, as a technique called backpropagation. The weights and biases are updated using the gradients of the loss function with respect to each weight and bias. This process is repeated iteratively until the error is minimized.

2.2. Overview of PSO Optimization

Particle Swarm Optimization (PSO) is a global computational method that optimizes a problem by iteratively trying to improve a candidate solution with regard to a given measure of quality. It is inspired by the social behavior of organisms such as bird flocking or fish schooling.

A population (or swarm) of candidate solutions (particles) is initialized randomly within the search space. Each particle's position (at every step of the algorithm) is evaluated using a fitness function, and keeps track of its own best position (**pbest**). Overall, the global best position (**gbest**) is tracked throughout the algorithm.

The positions of each particle are updated based on their current velocities and their own distance relative to both **pbest** and **gbest**. The fitness evaluation along the update are repeated until either a sufficiently good fitness is globally achieved or the max number of iteration is reached [3].

2.2.1. Mathematical formulation

The velocity and position updates are typically performed using the following equations [3]:

Velocity Update:

$$\Delta\theta_{i(t+1)} = w \cdot \Delta\theta_{i(t)} + c_1 \cdot r_p \cdot \left(p_{b,i} - \theta_{i(t)}\right) + c_2 \cdot r_g \cdot \left(g_b - \theta_{i(t)}\right)$$

Position Update:

$$\theta_{i(t+1)} = \theta_{i(t)} + \Delta\theta_{i(t+1)}$$

Where:

- $\Delta\theta(t)$ is the velocity of particle i at time step t.
- $\theta_i(t)$ is the position of particle i at time step t.
- w is the inertia weight, which controls the impact of the previous velocity on the current velocity. Higher inertia calls for exploration (finding the best zone) and lower for exploitation (fine-tuning the parameters).
- c_1 and c_2 are acceleration coefficients that control how much the particle is directed towards pbest and gbest positions. c_1 controls the attraction towards pbest, c_2 the attraction for gbest.
- r_p and r_g are random numbers between 0 and 1.
- p_b is the personal best position of particle i.
- g_b is the global best position found.

3. Implementation

3.1. Data Preparation

Data used for this assignement, relating to breast cancer, is contained in *data.mat*. For further training, the data is normalized via the following methods in MATLAB :

```
function [mind, maxd, standard_data] = standardizeData(data)
    mind = min(data); % No min/max over the dataset
    maxd = max(data); % in order to see dynamics within the vector
    standard_data = (data-mind)/(maxd-mind);
end

function destdData = deStandardizeData(mind, maxd, data)
    destdData = data.*(maxd' - mind') + mind';
end
```

3.2. Classification

This assignement is about classification of data. The classification is implemented the following way:

```
function classifiedData = classifyData(data, b0, b1)
    classifiedData = data;
    classifiedData(classifiedData < (b0+b1)/2) = b0;
    classifiedData(classifiedData >= (b0+b1)/2) = b1;
end
```

Every output must be either 2 or 4, with the boundary being 3:

- If they are above 3, they are classified at 4.
- If they are below 3, they are classified at 2.

3.3. Neural Network Setup

```
k = 20; % Number of k-subsets

% Creates a cross-validation partition object
cv = cvpartition(size(data, 1), 'KFold', k);

% Number of neurons in the hidden layer
hiddenLayerSize = round(length(data(1, 2:end)));

% Creates the network
net = feedforwardnet(hiddenLayerSize);

% Configures the network
net.trainFcn = 'trainlm';  % Levenberg-Marquardt backpropagation
net.performFcn = 'mse';    % Mean Squared Error performance function
```

As previously said, the default configuration is the data splitted in a cross-validation partition, with the number of hidden layers automatically determined from the length of inputs. The network is a feedforward network, with Levenberg-Marquardt backpropagation and Mean Squared-Error as the performance evaluation.

3.4. Neural Network training

```
% Train and evaluate the network
for i = 1:k
    % Get the training and test indices
    trainIdx = training(cv, i);
    testIdx = test(cv, i);

    % Spliting the data and labels
    X_train = data(trainIdx, 2:end);
    Y_train = data(trainIdx, 1);
    X_test = data(testIdx, 2:end);
    Y_test = data(testIdx, 1);

    % Trains the network
    [net, tr] = train(net, X_train', Y_train');

    % Evaluates the network
    outputs = net(X_test');
    performance = perform(net, Y_test', outputs);

    cumul_perf = [cumul_perf, performance];
    % Display the performance
    fprintf('Network performance: %.4f\n', performance);

    if (performance < best_perf)
        bestNet = net;
    end
end
```

For every subset available, we train over the k-1 remaining subsets using the *net()* function. In this implementation, the data is not averaged but selected as the best performance (on the basis of minimum MSE).

3.5. PSO Optimization

```
function global_best_position = pso_linear_optimization(dataset)
    % Parameters
    n_particles = 50; % Number of particles
    n_dimensions = 30; % Number of dimensions (inputs)
    max_iterations = 1000; % Maximum number of iterations
    w1 = 0.9; % Inertia weight initial
    w2 = 0.4; % Intertia weight final
    c1 = 1.7; % Cognitive coefficient
    c2 = 1.75; % Social coefficient
    condition = 10e-2;

    % Initialize particles
    particles = rand(n_particles, n_dimensions);
    velocities = rand(n_particles, n_dimensions);
    personal_best_positions = particles;
    personal_best_scores = inf(n_particles, 1);

    % Initialize global best
    global_best_position = particles(1, :);
    global_best_score = inf;

    % Main loop
    for iteration = 1:max_iterations
        for i = 1:n_particles
```

```
            % Evaluates the current particle using the dataset
            current_score = linear_objective_function(particles(i, :), dataset);

            if current_score < condition
                break;
            end

            % Updates personal best
            if current_score < personal_best_scores(i)
                personal_best_scores(i) = current_score;
                personal_best_positions(i, :) = particles(i, :);

                % Updates global best
                if current_score < global_best_score
                    global_best_score = current_score;
                    global_best_position = particles(i, :);
                end
            end

            % Updates velocity and position
            r1 = rand(1, n_dimensions);
            r2 = rand(1, n_dimensions);
            velocities(i, :) = (w1 - (iteration/max_iterations)*(w1 - w2)) *
velocities(i, :) + ...
                               c1 * r1 .* (personal_best_positions(i, :) -
particles(i, :)) + ...
                               c2 * r2 .* (global_best_position
- particles(i, :));
            particles(i, :) = particles(i, :) + velocities(i, :);
        end

        % Display progress
        %fprintf('Iteration %d: Best Score = %f\n', iteration, global_best_score);
    end
    fprintf('Best score: %f\n', global_best_score);
end
```

For repeatability reasons, the PSO algorithm is implemented as a function. A small number of 100 particles is chosen to have a small processing time and to avoid overfitting. r1 = r2, meaning that the algorithm put an equal emphasis on global and personal bests.

```
function score = linear_objective_function(x, dataset)
    % Evaluates the fitness of a particle based on the dataset
    inputs = dataset(:, 2:end);
    actual_outputs = dataset(:, 1);
    predicted_outputs = classifyData(inputs * x', 2, 4); % Linear combination
of inputs
    errors = predicted_outputs - actual_outputs;
    score = mean(errors.^2); % Mean squared error
end
```

As PSO particle can be interpreted as linear parameters, we simply calculates the resulting output from dataset and parameters to compute the score. The data is classified for the training to match the classification we expect of the model. The error is calculated the same way the neural network does, with mean-squared error.

4. Results

The neural network and the PSO algorithm are trained via cross-validation, where the training set is used as training and the test signal as verification. The validation set is the whole dataset.

4.1. Feedforward neural network

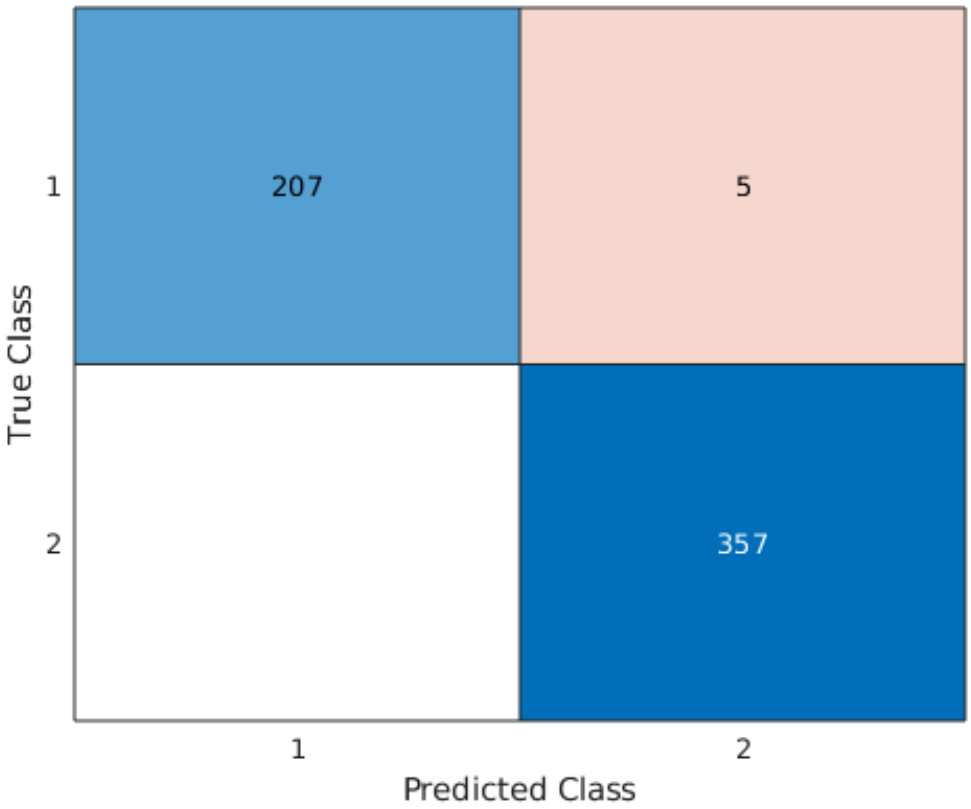


Figure 1: Neural network output confusion matrix. 1 = Malign, 2 = Benign

The neural network classifies correctly for 99.12% of the samples, the error being 5 samples being classified as benign while they were malign. Notably, there is no miss-classification for malign being benign.

Reference repartition: $37.2583m, 62.7417b$.
Output repartition: $36.3796m, 63.6204b$.

4.2. Particle Swarm Optimization

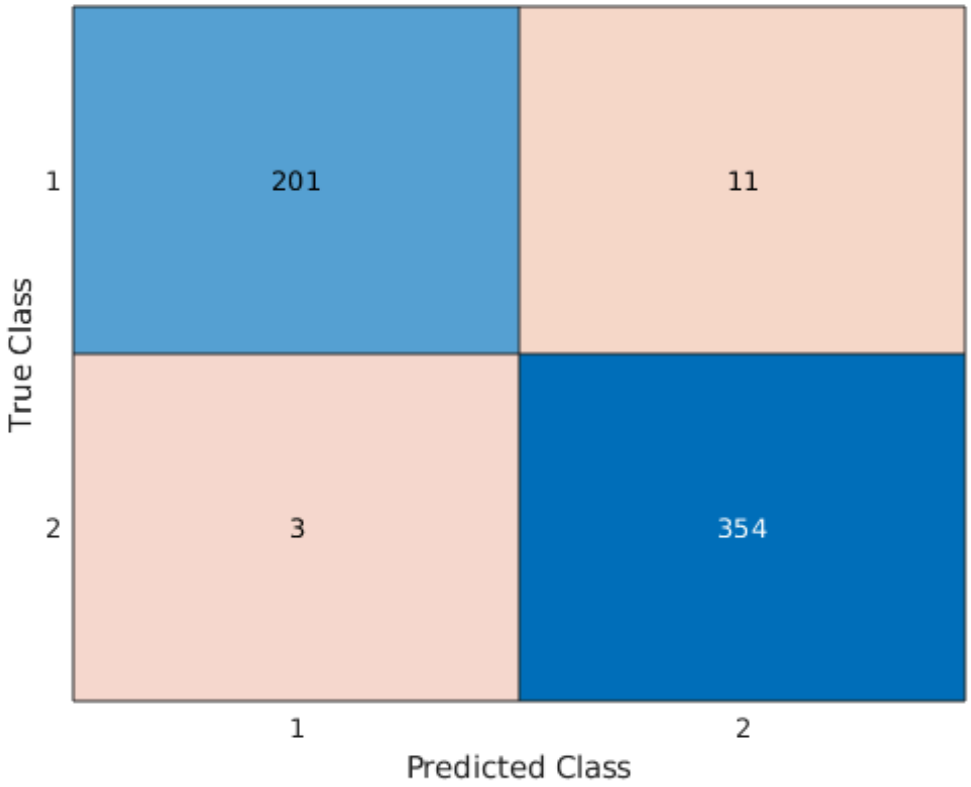


Figure 2: PSO output confusion matrix. 1 = Malign, 2 = Benign

The PSO output bears more or less the same characteristics than the neural network output, albeit with a very slightly worse classification % of 97.53 % (difference of 1.58%). Notably, it miss-classifies the 3 malign as benign. This slight difference can be attributed to tuning error.

Reference repartition: $37.2583\%m, 62.7417\%b$.
Output repartition: $35.6766\%m, 64.3234\%b$.

4.3. Linear activation neural network

4.3.1. Implementation

```
% Set the transfer function for the hidden layer to be linear
net.layers{1}.transferFcn = 'purelin'; % Linear transfer function for the hidden
layers

% Set the transfer function for the output layer to be linear
net.layers{2}.transferFcn = 'purelin'; % Linear transfer function for the outpout
layer
```

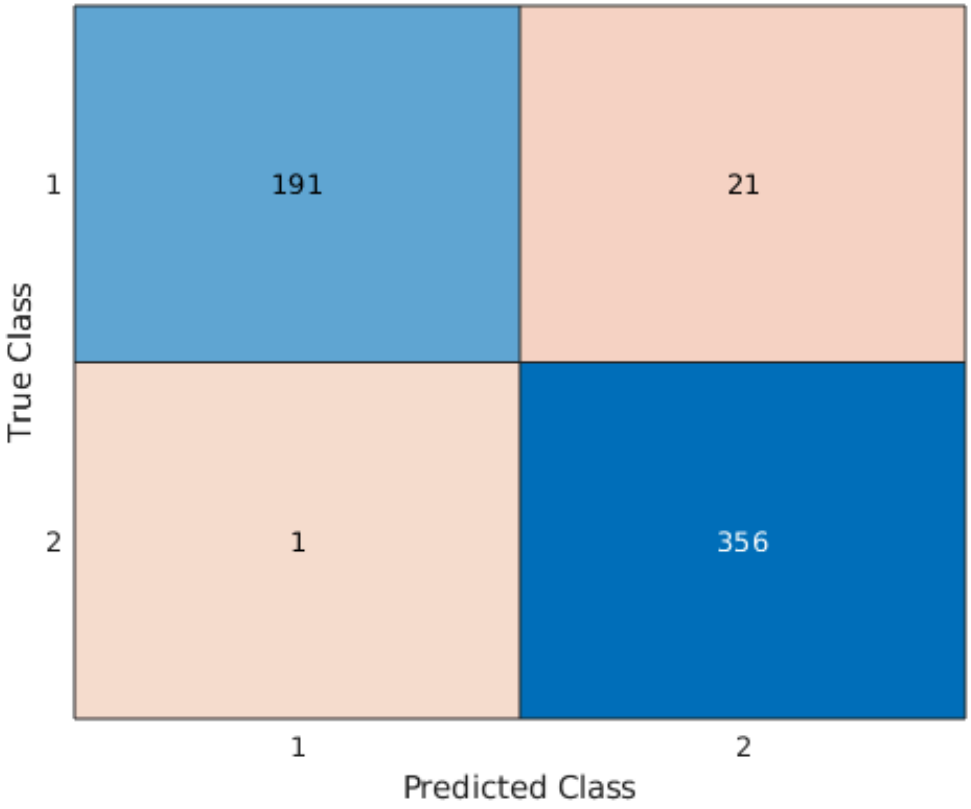


Figure 3: Neural network linear decision confusion matrix

When switching the transfer functions to linear transfer functions, the neural network becomes a linear classifier. This is because the composition of linear functions of each layers is still a linear function.

It is really close in behaviour to PSO while having a little worse classification of 96.309%. This can be considered as a good result and as such, switching the layers into linear functions still provides good results.

Reference repartition: $37.2583\%m, 62.7417\%b$.
Output repartition: $34.9736m, 65.0264\%b$.

5. Conclusion

The classification results show that the feedforward neural network achieves the highest accuracy, correctly classifying 99.12% of samples, with no false positives for malignant cases. PSO performs slightly worse at 97.53%, but still demonstrates strong classification capability with a simpler, linear model. A linearized neural network further confirms that linear methods can still be effective, achieving 96.31% accuracy.

Bibliography

[1] I. Škrjanc, “Neural network - PPT slides for the 10th lecture.” Apr. 2016.

[2] I. Škrjanc, “Optimizing the model complexity - PPT slides for the 7th lecture.” Apr. 2016.

[3] I. Škrjanc, “Global nonlinear optimization - PPT slides for the 5th lecture.” Mar. 2016.

Index of Figures

Figure 1 Neural network output confusion matrix. 1 = Malign, 2 = Benign 9
Figure 2 PSO output confusion matrix. 1 = Malign, 2 = Benign 10
Figure 3 Neural network linear decision confusion matrix 11